

Lista 1 – INF05008

Instruções

- USE OS NOMES DE TIPOS DE DADOS E FUNÇÕES DEFINIDOS NAS QUESTÕES.
- Use o **modelo (template) da solução disponível no Moodle**.
- TODAS as funções, inclusive as auxiliares, devem ter documentação completa: contrato, objetivo, e pelo menos 2 exemplos e 2 testes (só não precisa incluir testes nas funções que geram imagens). Se quiser, pode usar os testes como exemplos, mas neste caso isso deve ser indicado (leia o texto sobre exemplos e testes).
- Muitas vezes não é dito explicitamente no enunciado que um tipo deve ser definido, faz parte da solução identificar todos os tipos necessários para a realização dos exercícios.
- Siga as instruções para a realização de exercícios disponível no Moodle (no item *Informações gerais*).
- **Em nenhum exercício sua solução pode ter mais do que 7 ou 8 linhas de código por função (sem considerar linhas de comentários).**
- Nesta lista você **deve usar somente os comandos (funções) vistos em aula** até o laboratório 2, exceto na questão 7, na qual você pode usar também funções do pacote 2htdp/image. Ou seja, você **NÃO deve usar** listas, recursão, definições locais, alta-ordem, etc.

No Laboratório 2 já realizamos alguns exercícios com cartas de UNO. Algumas constantes, tipos e funções daquele laboratório estão incluídas no template para serem reutilizadas nas questões a seguir.

1. Construa a função **mesmo-num-ou-simb** que, dadas 2 cartas UNO, devolve 1 se tiverem o mesmo número ou símbolo e 0 caso contrário. Cartas curinga contam como qualquer número ou símbolo (ou seja, se uma das cartas for um curinga, deve ser retornado 1).
2. Imagine que queremos comparar as cartas da mão de um jogador com uma outra carta para saber quantas cartas de um determinado tipo há na mão do jogador. Desenvolva a função **número-cartas-iguais-mão** que recebe uma mão e uma carta UNO e devolve o número de cartas da mão que tem o mesmo tipo da carta passada como argumento. Curingas devem ser considerados como cartas de qualquer tipo.
3. Defina um tipo de dados chamado **Jogador**, que deve registrar o nome, a mão do jogador e os números de vitórias e derrotas deste jogador. Defina, no mínimo, 4 constantes cujos valores sejam do tipo **Jogador**. Construa uma função chamada **pontuação-jogador** que, dado um jogador, devolve sua pontuação total, considerando que cada vitória conta 10 pontos e cada derrota desconta 10 pontos.
4. Imagine que um jogador acabou de ganhar uma partida. Assim, seu número de vitórias deve ser atualizado. Construa a função **atualiza-vitorias-jogador** que, dado um jogador, atualiza o número de vitórias do jogador, somando um ao número de vitórias que ele já tem (devolvendo um registro novo do jogador com o número de vitórias atualizado).
5. Defina uma estrutura de dados chamada **Dupla** para representar duplas de jogadores, sabendo que é importante registrar o nome da dupla, seu emblema (uma imagem) e os jogadores que fazem parte da dupla. Defina, no mínimo, 2 constantes cujos valores sejam do tipo **Dupla**. Desenvolva, também, a função **melhor-dupla** que recebe 2 duplas e diz qual tem a maior pontuação, considerando as pontuações de seus dois jogadores. O resultado deve ser o registro da dupla vencedora ou a string "Empate", caso as duplas estejam empatadas.
6. Quando o torneio termina, deve ser definida a dupla vencedora. Porém, antes dessa definição, deve ser verificado se os registros das duplas estão consistentes: neste caso, iremos verificar se a soma dos números de vitórias é igual à soma do número de derrotas (imaginando que em cada rodada do jogo, haverá apenas um jogador vencedor e um que ficará em último lugar, sendo considerado o perdedor). Construa uma função chamada **define-vencedora** que, dadas 2 duplas, gera um registro com a dupla vencedora e seu número de pontos. Se houver empate, o registro deve conter a string "Empate" e o número de pontos. Porém, se houver inconsistência nos registros das duplas, deve ser retornada a mensagem **"Inconsistência detectada: número de vitórias diferente do número de derrotas"**. *Note que esta função pode devolver um par (com uma dupla ou string e uma a pontuação, nesta ordem) ou uma string. Portanto, você deve definir os tipos necessários. O tipo do resultado da aplicação da função deve se chamar **Resultado**, e o tipo da estrutura que contém o par nome e mesa deve se chamar **Par**.*

7. Desenvolva a função **mostra-duplas** que, dadas 2 duplas, gera uma imagem mostrando os nomes de todos os jogadores e suas cartas, mostrando qual dupla venceu o jogo. Ver exemplos abaixo. Requisitos:

- Os nomes e cartas de todos os jogadores devem ser mostrados.
- Os nomes e emblemas das duplas devem ser mostrados.
- Você **deve** redefinir a função **desenha-carta** que fizemos em aula para fazer um desenho diferente das cartas de UNO.
- Não é permitido usar imagens prontas, você deve construir as representação visuais de suas cartas e mesa usando o pacote de imagens `2http/image` do Racket, <https://docs.racket-lang.org/teachpack/2httpimage.html>.
- O código deve ser organizado e legível (lembre que cada função não pode ter mais que 8 linhas).

Dica: Decomponha o problema em problemas menores e construa a solução através da composição das soluções dos problemas menores.

Preste atenção às instruções. Um dos objetivos desta questão é exercitar a construção de soluções usando apenas primitivas básicas, portanto não é permitido usar comandos que não foram vistos em aula até o laboratório 2.

O grande desafio é resolver o problema com o código mais legível possível! Portanto, a legibilidade do código (considerando organização, escolha de nomes, decomposição) fará parte da avaliação. Também será avaliada a criatividade no desenho das cartas e a apresentação das duplas.

Dupla D1								Dupla D2							
Jogador J1				Jogador J2				Jogador J3				Jogador J4			
															
Vitórias: 0 Derrotas: 0				Vitórias: 3 Derrotas: 2				Vitórias: 0 Derrotas: 2				Vitórias: 4 Derrotas: 3			
Pontos: 0				Pontos: 10				Pontos: -20				Pontos: 10			

Dupla vencedora: D1 (10 pontos)